

Name : Riya Dinesh Paturde.

Batch: B2

Roll No: B-24

Aim: Numerical Computation with Numpy: perform different computation using Numpy library.

Theory:

1) Explain the role of Numpy in the python scientific computing ecosystem.

Ans: NumPy (short for Numerical Python) is a foundational library in the Python scientific computing ecosystem. It provides the core tools for working with large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on them efficiently.

Role of NumPy

1. Efficient Multidimensional Arrays

Supports 1D, 2D, and n-dimensional arrays Much faster and more memory-efficient than Python lists.

2. Mathematical and Statistical Functions

Linear algebra (dot products, matrix multiplication).

Statistics (mean, median, standard deviation).

3. Foundation for Other Libraries

Many major scientific libraries depend on NumPy arrays as their core data structure:

SciPy → advanced scientific algorithms Pandas → tabular data handling Matplotlib → data visualization Scikit-learn → ML algorithms

2) Why is NumPy considered faster than pure Python computations?

Ans: NumPy is considered faster than pure Python mainly because it avoids Python's biggest performance bottlenecks and uses highly optimized low-level implementations.

NumPy is faster than pure Python computations because:

1. Uses compiled code (C/C++) NumPy operations are implemented in optimized compiled languages like C, so they run much faster than Python's interpreted code.

2. Vectorization NumPy performs operations on entire arrays at once instead of element-by-element loops, reducing Python overhead.

3. Efficient memory usage NumPy arrays store elements of the same type in contiguous memory, which improves access speed and cache efficiency.

4. No dynamic type checking Unlike Python lists, NumPy arrays have fixed data types, so there's no need to check the type of each element during computation.

3) What types of mathematical operations can be performed using NumPy?

Ans: 1. Arithmetic Operations

These are basic operations applied element-wise on arrays:

Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%), Power (**).

2. Algebraic (Linear Algebra) Operations

Used for matrix and vector calculations:

Matrix multiplication (dot() or @), Determinant (linalg.det), Inverse of a matrix (linalg.inv).

3. Statistical Operations

Useful for data analysis:

Mean (mean), Median (median), Standard deviation (std), Variance (var), Minimum and maximum (min, max).

4. Trigonometric Functions

Used in geometry and scientific calculations:

Sine (sin), Cosine (cos), Tangent (tan).

4) Explain indexing and slicing in NumPy arrays.

Ans: Indexing Indexing in NumPy is the method of accessing individual elements of an array using their position.

Each element in an array has an index number, starting from 0.

In one-dimensional arrays, a single index is used.

In multi-dimensional arrays, multiple indices are used (e.g., row and column).

Negative indexing is also allowed, where -1 refers to the last element.

2.Slicing

Slicing is used to extract a group (subset) of elements from an array.

It selects a range of elements based on a defined start, stop, and step.

In multi-dimensional arrays, slicing can be applied across rows and columns simultaneously.

Slicing returns a view of the original array, not a separate copy.

5)How does NumPy integrate with libraries like:

a.Pandas b.Matplotlib c.Scikit- learn

Ans: a.Integration with Pandas

Pandas is built on top of NumPy.

It uses NumPy arrays internally to store data efficiently.

Data structures like Series and DataFrame are based on NumPy arrays.

NumPy provides fast numerical operations that Pandas uses for data manipulation.

b.Integration with Matplotlib Matplotlib uses NumPy arrays as input for plotting graphs.

Data for plots (lines, bars, histograms, etc). is stored and processed using NumPy arrays.

NumPy helps in performing mathematical transformations before visualization.

c.Integration with Scikit-learn

Scikit-learn uses NumPy arrays to represent datasets (features and labels).

All machine learning models in Scikit-learn accept NumPy arrays as input.

Numerical computations like distance calculations, matrix operations, and model training rely on NumPy.

```
import numpy as np

# Creating an array
a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])

print("Array A:", a)
print("Array B:", b)

# Arithmetic operations
print("\nArithmetic Operations:")
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)

# Statistical operations
print("\nStatistical Operations:")
print("Mean of A:", np.mean(a))
print("Median of A:", np.median(a))
print("Standard Deviation of A:", np.std(a))

# Aggregate operations
print("\nAggregate Operations:")
print("Sum of A:", np.sum(a))
print("Product of A:", np.prod(a))

# Trigonometric operations
print("\nTrigonometric Operations:")
angles = np.array([0, 30, 45, 60, 90])
print("Sine values:", np.sin(np.radians(angles)))

# Exponential and logarithmic operations
print("\nExponential and Logarithmic Operations:")
print("Exponential:", np.exp(b))
print("Log (natural):", np.log(a))

# Rounding operations
c = np.array([1.2, 2.6, 3.5, 4.9])
print("\nRounding Operations:")
print("Round:", np.round(c))
print("Floor:", np.floor(c))
print("Ceil:", np.ceil(c))
```

```
# Maximum and Minimum
print("\nMaximum and Minimum:")
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))

#Indexing and Slicing
print("\nIndexing and Slicing:")
print("First element of A:",a[0])
print("last element of A:", a[-1])

#Sorting
print("\nSorting:")
print("Sorted A:", np.sort(a))
```

```
Array A: [10 20 30 40 50]
Array B: [1 2 3 4 5]

Arithmetic Operations:
Addition: [11 22 33 44 55]
Subtraction: [ 9 18 27 36 45]
Multiplication: [ 10  40  90 160 250]
Division: [10. 10. 10. 10. 10.]

Statistical Operations:
Mean of A: 30.0
Median of A: 30.0
Standard Deviation of A: 14.142135623730951

Aggregate Operations:
Sum of A: 150
Product of A: 12000000

Trigonometric Operations:
Sine values: [0.          0.5          0.70710678  0.8660254   1.          ]

Exponential and Logarithmic Operations:
Exponential: [ 2.71828183  7.3890561   20.08553692  54.59815003 148.4131591 ]
Log (natural): [2.30258509 2.99573227 3.40119738 3.68887945 3.91202301]

Rounding Operations:
Round: [1. 3. 4. 5.]
Floor: [1. 2. 3. 4.]
Ceil: [2. 3. 4. 5.]

Maximum and Minimum:
Maximum: 50
Minimum: 10

Indexing and Slicing:
First element of A: 10
last element of A: 50

Sorting:
Sorted A: [10 20 30 40 50]
```

Conclusion

This NumPy program demonstrates how numerical data can be efficiently stored and analyzed using arrays. It shows important operations such as calculating total marks, average marks, subject-wise performance, highest and lowest marks, and identifying the topper of the class.

The program also shows how NumPy helps in quickly finding the class topper using built-in functions like `sum()`, `mean()`, `max()`, `min()`, and `argmax()` without using complex loops.

